

TSC Instruction Set Architecture

- Note that it is not a full description of TSC ISA

1 Instruction Format

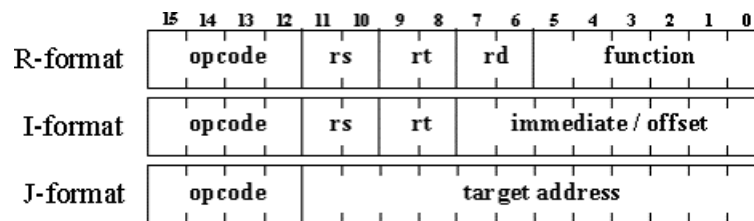


Figure 1: Instruction Format

2 Instruction Opcodes

Instruction	Opcode	Function Code	Format
ADD	15	0	R
SUB	15	1	R
AND	15	2	R
ORR	15	3	R
NOT	15	4	R
TCP	15	5	R
SHL	15	6	R
SHR	15	7	R
ADI	4	-	I
ORI	5	-	I
LHI	6	-	I
RWD	15	27	R
WWD	15	28	R
LWD	7	-	I
SWD	8	-	I
BNE	0	-	I
BEQ	1	-	I
BGZ	2	-	I
BLZ	3	-	I
JMP	9	-	J
JAL	10	-	J
JPR	15	25	R
JRL	15	26	R
HLT	15	29	R

3 Instruction Description

3.1 ADD

3.1.1 Assembler Format

ADD \$rd, \$rs, \$rt

3.1.2 Description

The contents of \$rs and the contents of \$rt are added to form the result. The result is placed into \$rd

3.1.3 Operation

$\$rd \leftarrow \$rs + \$rt$

3.1.4 Examples

ADD \$1, \$2, \$0 ;	$\$3 \leftarrow \$0 + \$1$
ADD \$2, \$2, \$2 ;	$\$2 \leftarrow \$2 + \$2$

3.2 SUB

3.2.1 Assembler Format

SUB \$rd, \$rs, \$rt

3.2.2 Description

The contents of \$rt are subtracted from the contents of \$rs to form the result. The result is placed into \$rd

3.2.3 Operation

$\$rd \leftarrow \$rs - \$rt$

3.2.4 Examples

SUB \$3, \$0, \$1 ;	$\$3 \leftarrow \$0 - \$1$
SUB \$0, \$0, \$0 ;	$\$0 \leftarrow \$0 - \$0$ This clears \$0

3.3 AND

3.3.1 Assembler Format

AND \$rd, \$rs, \$rt

3.3.2 Description

The contents of \$rs are combined with the contents of \$rt in a bit-wise logical AND operation. The result is placed into \$rd

3.3.3 Operation

$\$rd \leftarrow \$rs \& \$rt$

3.3.4 Examples

AND \$0, \$1, \$2 ;	$\$0 \leftarrow \$1 \& \$2$
AND \$3, \$3, \$3 ;	$\$3 \leftarrow \$3 \& \$3$ Contents of \$3 are unchanged

3.4 ORR

3.4.1 Assembler Format

ORR \$rd, \$rs, \$rt

3.4.2 Description

The contents of \$rs are combined with the contents of \$rt in a bit-wise logical OR operation.

The result is placed into \$rd

3.4.3 Operation

$\$rd \leftarrow \$rs - \$rt$

3.4.4 Examples

ORR \$1, \$2, \$1 ; $\$1 \leftarrow \$2 - \$1$
ORR \$3, \$3, \$3 ; $\$3 \leftarrow \$3 - \$3$ Contents of \$3 are unchanged

3.5 NOT

3.5.1 Assembler Format

NOT \$rd, \$rs

3.5.2 Description

The bit-wise inverse of the contents of \$rs are placed into \$rd

3.5.3 Operation

$\$rd \leftarrow !\rs

3.5.4 Examples

NOT \$0, \$1; $\$0 \leftarrow !\1
NOT \$3, \$3; $\$3 \leftarrow !\3

3.6 TCP

3.6.1 Assembler Format

TCP \$rd, \$rs

3.6.2 Description

The two's complement of the contents of \$rs is placed into \$rd

3.6.3 Operation

$\$rd \leftarrow !\$rs + 1$

3.6.4 Examples

TCP \$0, \$2; $\$0 \leftarrow !\$2 + 1$
TCP \$1, \$1; $\$1 \leftarrow !\$1 + 1$

3.7 SHL

3.7.1 Assembler Format

SHL \$rd, \$rs

3.7.2 Description

The contents of \$rs are shifted left one bit, inserting zero into the least significant bit. The result is placed into \$rd

3.7.3 Operation

$\$rd \leftarrow !\$rs14..0 \text{ \#} 0$

3.7.4 Examples

SHL \$0, \$1; $\$0 \leftarrow \$1 << 1$

3.8 SHR

3.8.1 Assembler Format

SHR \$rd, \$rs

3.8.2 Description

The contents of \$rs are shifted right one bit, sign-extending the most significant bit i.e. the value of the most significant bit is preserved. The result is placed into \$rd

3.8.3 Operation

$$\$rd \leftarrow \$rs15 \text{ \#\# } \$rs15..1$$

3.8.4 Examples

SHR \$2, \$1;	$\$2 \leftarrow \$1 \gg 1$
SHR \$1, \$1;	$\$1 \leftarrow \$1 \gg 1$

3.9 ADI

3.9.1 Assembler Format

ADI \$rt, \$rs, imm

3.9.2 Description

The 8-bit immediate value is sign-extended to 16 bits and added to the contents of \$rs to form the result. The result is placed into \$rt

3.9.3 Operation

$$\$rt \leftarrow \$rs + \{ (imm7)8 \text{ \#\# } imm7..0 \}$$

3.9.4 Examples

ADI \$0, \$1, -17 ;	$\$0 \leftarrow \$1 - 17$
---------------------	---------------------------

3.10 ORI

3.10.1 Assembler Format

ORI \$rt, \$rs, imm

3.10.2 Description

The 8-bit immediate value is zero-extended to 16 bits and combined with the contents of \$rs in a bit-wise logical OR operation. The result is placed into \$rt.

3.10.3 Operation

$$\$rt \leftarrow \$rs \text{ \textasciitilde\ } \{ 08 \text{ \#\# } imm7..0 \}$$

3.10.4 Examples

ORI \$0, \$1, 0xff ;	$\$0 \leftarrow \$1 \text{ \textasciitilde\ } 0x00ff$
ORI \$1, \$1, 1000 ;	Is this valid? Why or why not?
ORI \$2, \$3, 0 ;	$\$2 \leftarrow \3 ; Contents of \$3 are copied to \$2

3.11 LHI

3.11.1 Assembler Format

LHI \$rt, imm

3.11.2 Description

The 8-bit immediate value is concatenated to 8 bits of zeros, the immediate value being the most significant halfword. The result is placed into \$rt

3.11.3 Operation

$$\$rt \leftarrow imm7..0 \text{ \#\# } 08$$

3.11.4 Examples

LHI \$3, 15 ;	$\$3 \leftarrow 15 \ll 8$
LHI \$2, 0 ;	$\$2 \leftarrow 0$ This clears \$2

3.12 RWD

3.12.1 Assembler Format

RWD \$rd

3.12.2 Description

Program execution is halted until an input signal is received on the input port. A 16-bit word is read from the input port and placed into \$rd

3.12.3 Operation

$\$rd \leftarrow \text{inputport}$

3.12.4 Examples

RWD \$2 ; $\$2 \leftarrow \text{input port}$

3.13 WWD

3.13.1 Assembler Format

WWD \$rs

3.13.2 Description

The contents of \$rs are written to the output port and the output signal is asserted. Execution stops until an acknowledge is received from the output device

3.13.3 Operation

$\text{output port} \leftarrow \rs

3.13.4 Examples

WWD \$0 ; $\text{output port} \leftarrow \0

3.14 LWD

3.14.1 Assembler Format

LWD \$rt, \$rs, offset

3.14.2 Description

The 8-bit address offset is sign-extended and added to the contents of \$rs to form a memory address. The word at the specified memory location is loaded into \$rt

3.14.3 Operation

$\$rt \leftarrow M[\$rs + \{ (\text{offset}7)8 \# \text{offset}7..0 \}]$

3.14.4 Examples

LWD \$0, \$0, 16;	$\$0 \leftarrow M[\$0 + 16]$
LWD \$0, \$3, -4;	$\$0 \leftarrow M[\$3 - 4]$
LWD \$0, \$3, 0;	$\$0 \leftarrow M[\$3]$

3.15 SWD

3.15.1 Assembler Format

SWD \$rt, \$rs, offset

3.15.2 Description

The 8-bit address offset is sign-extended and added to the contents of \$rs to form a memory address. The contents of \$rt are stored at the specified memory location

3.15.3 Operation

$M[\$rs + \{ (\text{offset}7)8 \# \text{offset}7..0 \}] \leftarrow \rt

3.15.4 Examples

SWD \$1, \$1, 120;	$M[\$1 + 120] \leftarrow \1
SWD \$1, \$3, -2;	$M[\$3 - 2] \leftarrow \1
SWD \$1, \$0, 0;	$M[\$3] \leftarrow \1

3.16 BNE

3.16.1 Assembler Format

BNE \$rs, \$rt, offset

3.16.2 Description

A branch target address is computed from the sum of the address of the instruction after the branch instruction and the 8-bit, sign-extended offset. The contents of \$rs and \$rt are compared. If they are not equal, then the target address is written into the PC and program execution continues with the instruction at the target address. Otherwise, program execution continues with the instruction following the branch

3.16.3 Operation

If $\$rs \neq \rt then $\$pc \leftarrow \$pc + \{ (\text{offset}7)8 \text{ ## offset}7..0 \}$

3.16.4 Examples

BNE \$0, \$2, 6;	If $\$0 \neq \2 then $\$pc \leftarrow \$pc + 7$
BNE \$1, \$2, LOOP1	If $\$1 \neq \2 then $\$pc \leftarrow \text{LOOP1}$

The last example is the common usage.

Note that if a label is in the offset field in the assembly instruction, the assembler will compute the offset and insert it into the binary instruction. If a number is in the offset field, the number itself will be inserted into the binary instruction

3.17 BEQ

3.17.1 Assembler Format

BEQ \$rs, \$rt, offset

3.17.2 Description

A branch target address is computed from the sum of the address of the instruction after the branch instruction and the 8-bit, sign-extended offset. The contents of \$rs and \$rt are compared. If they are equal, then the target address is written into the PC and program execution continues with the instruction at the target address. Otherwise, program execution continues with the instruction following the branch

3.17.3 Operation

If $\$rs == \rt then $\$pc \leftarrow \$pc + \{ (\text{offset}7)8 \text{ ## offset}7..0 \}$

3.17.4 Examples

BEQ \$0, \$3, 6;	If $\$0 == \3 then $\$pc \leftarrow \$pc + 6$
BEQ \$2, \$0, -36;	If $\$2 == \0 then $\$pc \leftarrow \$pc - 36$
BNE \$0, \$2, BYE	If $\$0 == \2 then $\$pc \leftarrow \text{BYE}$

The last example is the common usage.

Note that if a label is in the offset field in the assembly instruction, the assembler will compute the offset and insert it into the binary instruction. If a number is in the offset field, the number itself will be inserted into the binary instruction.

3.18 BGZ

3.18.1 Assembler Format

BGZ \$rs, offset

3.18.2 Description

A branch target address is computed from the sum of the address of the instruction after the branch instruction and the 8-bit, sign-extended offset. The contents of \$rs and zero are compared. If the contents of \$rs are greater than zero, then the target address is written into the PC and program execution continues with the instruction at the target address. Otherwise, program execution continues with the instruction following the branch

3.18.3 Operation

If \$rs $\geq$ 0 then $\$pc \leftarrow \$pc + \{ (\text{offset}7)8 \text{ ## offset}7..0 \}$

3.18.4 Examples

BGZ \$3, 42;	If \$3 $\geq$ 0 then $\$pc \leftarrow \$pc + 43$
BGZ \$2, -3;	If \$2 $\geq$ 0 then $\$pc \leftarrow \$pc - 2$
BGZ \$0, N2	If \$0 $\geq$ 0 then $\$pc \leftarrow N2$

The last example is the common usage.

Note that if a label is in the offset field in the assembly instruction, the assembler will compute the offset and insert it into the binary instruction. If a number is in the offset field, the number itself will be inserted into the binary instruction

3.19 BLZ

3.19.1 Assembler Format

BLZ \$rs, offset

3.19.2 Description

A branch target address is computed from the sum of the address of the instruction after the branch instruction and the 8-bit, sign-extended offset. The contents of \$rs and zero are compared. If the contents of \$rs are less than zero, then the target address is written into the PC and program execution continues with the instruction at the target address. Otherwise, program execution continues with the instruction following the branch

3.19.3 Operation

If \$rs $<$ 0 then $\$pc \leftarrow \$pc + \{ (\text{offset}7)8 \text{ ## offset}7..0 \}$

3.19.4 Examples

BGZ \$3, 200;	If \$3 $\geq$ 0 then $\$pc \leftarrow \$pc + 200$
BGZ \$2, -17;	If \$2 $\geq$ 0 then $\$pc \leftarrow \$pc - 16$
BGZ \$0, JEFF	If \$0 $\geq$ 0 then $\$pc \leftarrow JEFF$

The last example is the common usage.

Note that if a label is in the offset field in the assembly instruction, the assembler will compute the offset and insert it into the binary instruction. If a number is in the offset field, the number itself will be inserted into the binary instruction

3.20 JMP

3.20.1 Assembler Format

JMP target

3.20.2 Description

A target address is computed by concatenating the four high-order bits of the instruction's address with the 12-bit, unsigned target offset. The target address is written into the PC and program execution continues with the instruction at the target address

3.20.3 Operation

$\$pc \leftarrow \$pc15..12 \text{ \#\# target}11.0$

3.20.4 Examples

JMP LOOP $\$pc \leftarrow \text{LOOP}$ If four high-order bits of $\$pc$ are 0
JMP 0x0100, -17; $\$pc \leftarrow 0xc100$ If $\$pc$ previously contained 0xc68f
JMP 0xeeff; Is this valid? No, it's not. Why not?

3.21 JAL

3.21.1 Assembler Format

JAL target

3.21.2 Description

The address of the next instruction is placed in $\$2$. A target address is computed by concatenating the four high-order bits of the instruction's address with the 12-bit, unsigned target offset. The target address is written into the PC and program execution continues with the instruction at the target address

3.21.3 Operation

$\$2 \leftarrow \pc $\$pc \leftarrow \$pc15..12 \text{ \#\# target}11.0$

3.21.4 Examples

JAL LOOP $\$2 \leftarrow \pc $\$pc \leftarrow \text{LOOP}$ If four high-order bits of $\$pc$ are 0
JAL 0x05b0, -17; $\$2 \leftarrow \pc $\$pc \leftarrow 0x91ae$ If $\$pc$ previously contained 0x91ae
JAL 0xff34; Is this valid? No, why not?

3.22 JPR

3.22.1 Assembler Format

JPR $\$rs$

3.22.2 Description

The contents of $\$rs$ are written to the PC. Program execution continues with the instruction at that address

3.22.3 Operation

$\$pc \leftarrow \rs

3.22.4 Examples

JPR $\$2$ $\$pc \leftarrow \2
JPR is commonly used in conjunction with JAL to return from procedure calls. JPR and JRL are the only control instructions that allow branching to any address in physical memory

3.23 JRL

3.23.1 Assembler Format

JRL $\$rs$

3.23.2 Description

The address of the next instruction is placed in $\$2$. The contents of $\$rs$ are written to the PC. Program execution continues with the instruction at that address

3.23.3 Operation

$\$2 \leftarrow \pc $\$pc \leftarrow \rs

3.23.4 Examples

JRL \$3 $\$2 \leftarrow \pc $\$pc \leftarrow \3

JRL \$2 What happens when this is executed?

JRL is commonly used to jump to procedures that are out of range for a JAL instruction. JPR and JRL are the only control instructions that allow branching to any address in physical memory.

3.24 HLT

3.24.1 Assembler Format

HLT

3.24.2 Description

Indicates the end of a program. When executed, the machine does not fetch

3.24.3 Operation

The machine halts

3.24.4 Examples

HLT